Ring Attention: Sequence Parallelism on Multi-GPU Clusters

Édouard Rabasse[†] Hippolyte Wallaert[†]

Abstract

Expanding the context window of large language models is essential for unlocking long-horizon agentic capabilities. However, the memory footprint of key-value states and attention activations scales linearly with sequence length, quickly exceeding the physical memory limits of a single GPU. We reproduce Ring Attention [5], which lifts this ceiling through **sequence parallelism**. Sharding and rotating the key/value state across a ring of P GPUs allows the maximum context length to scale linearly with the GPU count. We present a standalone CUDA implementation of Ring Attention, optimized via MPI and NCCL for bandwidth-limited, hierarchical clusters. Guided by a companion communication model, we investigate zigzag token assignment for causal load balancing and a hierarchical 2D ring topology that minimizes slow inter-node communication.

h1ppox99/ring-attention Animated figures

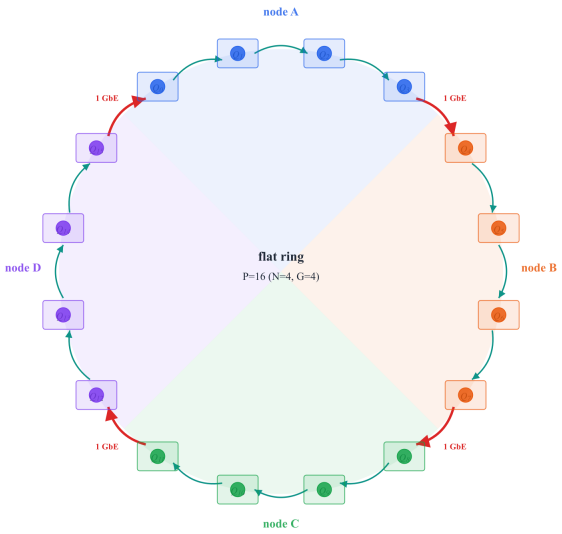


Figure 1: **Ring Attention**: GPUs hold K/V shards and rotate them around the ring.

[†]Stanford University, ICME; {hippowal, erabasse}@stanford.edu

1 Introduction

Problem description. Many parallelism strategies have been developed to scale model training and inference to thousands of GPUs, including data, tensor, pipeline or expert parallelism [6]. These techniques reduce heavily the memory requirements per GPU with regards to weights and activations, but memory requirements are also linear in the input sequence length. These sequence lengths have grown up to millions of tokens for the most powerful models as they are built to tackle long-horizon agentic tasks. Therefore, **context parallelism** resolves this issue by splitting input along sequence dimension. This does not affect MLP or LayerNorm computations but poses problem for computing attention blocks [7]. Indeed every token needs to access the key/value pairs of all the other tokens in the sequence (or preceeding tokens if causal). **Ring Attention** [5] solves this issue by performing optimized communications. Using Ring Attention allows to linearly scale the maximum input length with the number of processors available.

Formally the problem sits at the **trade-off between maximizing the sequence length being processed on a given computing cluster, and maximizing the speed of calculations**. For simplicity purposes, this report focuses on standard multi-head attention. [1]

Algorithms and state of the art Formally, let $Q, K, V \in \mathbb{R}^{S \times HD}$ denote the query, key, and value matrices, where S is the sequence length, H is the number of heads, and D is the head dimension. Attention maps these inputs to an output matrix $O \in \mathbb{R}^{S \times HD}$ via:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{D}} + M\right), \quad O = AV, \tag{1}$$

where $M \in \overline{\mathbb{R}}^{S \times S}$ is the attention mask. In causal modeling, M is lower-triangular. To avoid materializing A in global memory, FlashAttention [3] processes attention in blocks, tracking running row-wise online softmax statistics (m_i, ℓ_i) in registers and shared memory.

Across P distributed devices, a global `All-Gather` would reconstruct full K/V matrices but demand an unsustainable $O(S)$ memory footprint per GPU. Ring Attention [5] instead shards Q, K, V evenly into size $\frac{S}{P}$ fragments. The K/V shards are asynchronously rotated along a circular network ring over $P-1$ steps, updating local (m, ℓ, O) states at each hop and keeping memory bounded at $O(S/P)$. Figure 1 illustrates this logic.

Under **causal masking**, contiguous block layouts create a severe load imbalance where early ranks sit idle. To resolve this, advanced token layouts [2, 6] interleave sequence indices across ranks to balance the causal workload. These refinements are discussed in more detail in Section 2.3.

References implementations are written in Pytorch or JAX [8, 9] and target high-bandwidth and homogeneous fabrics. Our work has to be seen as a custom implementation heavily optimized for the class computing cluster. It focuses on a standalone CUDA+MPI/NCCL framework featuring low-overhead `fp16` transit, a load-balanced zigzag partitioner, and a custom hierarchical 2D-ring topology to preserve parallel efficiency over bandwidth-limited inter-node links.

2 Approach

2.1 Parallelization strategy (CUDA + MPI)

Ring communications. Each MPI rank drives one GPU and owns the queries plus the K/V shard for its S/P tokens. The only data crossing the wire is one `fp16` K/V shard per step, sent

[1] The codebase also handles Grouped-Query Attention [1].

point-to-point to the next rank. The design choice that hides it is *overlap*: each step posts a non-blocking send/recv on a copy stream and at once launches the attention kernel on a compute stream against the previously received, double-buffered shard, so the two run concurrently and a step costs $\max(T_{\text{cmp}}, T_{\text{comm}})$ rather than their sum. Communications are not identical across the cluster though: intra-node communications move shards device-to-device with `ncclSend`/`ncclRecv` over PCIe, but the cluster has no inter-node GPUDirect, so node-crossing hops fall back to NCCL's TCP transport over the 1 GbE uplink.

Attention kernel. Each step is one flash-style online-softmax pass. The grid is (query-row tiles $\times$ heads $\times$ batch); every thread owns one query row, keeps its running (m, ℓ, O) in registers, and consumes K/V tiles staged into shared memory through coalesced `float4` loads. The one subtlety is the causal mask: because K/V arrive rotated (ring) and permuted (zigzag), a rank cannot read token ordering off the shard index, so we evaluate the mask from *global* token positions. This single hook lets one kernel serve every layout of Section 2.3 unchanged.

2.2 Performance model

We model a single attention layer (batch $B = 1$, H heads of dimension D, FP16 transit so $\sigma = 2\,\text{bytes/element}$) sharded over P GPUs grouped into $\lceil P/g \rceil$ nodes of g GPUs. Each rank owns S/P tokens of Q, K, V and the ring completes the attention in $P-1$ rotation steps. We split each step into a compute and a communication term and, since the two run on independent streams, charge their maximum: $T_{\text{step}} \approx \max(T_{\text{cmp}}, T_{\text{comm}})$. All measured constants are collected in Table 2 (App. A).

Compute. At each round a rank scores its S/P local queries against the incoming S/P-token K/V shard for

$$\text{FLOP}_{\text{round}} \;=\; 4\,(S/P)^2\,DH \;=\; \frac{4\,S^2 DH}{P^2}. \tag{2}$$

Under a causal mask, correct zig-zag token assignment (Section 2.3) keeps this balanced across ranks and halves it. At an effective tensor-core rate F^{σ}_{eff}, the round therefore costs $T_{\text{cmp}} = 2\,(S/P)^2 DH \,/\, F^{\sigma}_{\text{eff}}$ in the causal case.

Communication. Each round, every rank sends one K/V shard to its successor and receives one from its predecessor under the usual latency–bandwidth model

$$T_{\text{comm}} \;=\; \alpha \;+\; 2\,\beta\,(S/P)\,DH\,\sigma, \tag{3}$$

where (α, β) are the intra- or inter-node constants. Because the inter-node uplink is $\sim 50\times$ slower than the intra-node path (Table 2), a flat ring is governed by the inter-node pair (α', β'), which we take as the limiter.

Overlap. Communication is fully hidden when $T_{\text{comm}} \le T_{\text{cmp}}$, i.e.

$$\alpha' \;+\; 2\,\beta'\,\sigma\,DH\,\frac{S}{P} \;\le\; \frac{2\,DH}{F^{\sigma}_{\text{eff}}}\Big(\frac{S}{P}\Big)^2. \tag{4}$$

Cancelling $2DH$ and dropping the small-message floor α' leaves a crossover that depends *only* on tokens-per-GPU:

$$\boxed{\frac{S}{P} \;\gtrsim\; \beta'\,\sigma\,F^{\sigma}_{\text{eff}} \;\approx\; 8.5\times 10^{-9}\cdot 2\cdot 23\times 10^{12} \;\approx\; 3.9\times 10^{5}\ \text{tokens/GPU}.} \tag{5}$$

2.3 Algorithmic variants explored

Sequence partitions. We implemented and evaluated three distinct sequence layouts to dispatch work and tokens more evenly across ranks in the **causal** setting. Figure 2 illustrates the different layouts.



Figure 2: Comparison of different layouts for sequence partition and work balancing.

- **Contiguous [5]:** rank p owns the block $\left[\frac{p}{P}S, \frac{p+1}{P}S\right)$. Under causal masking this is maximally imbalanced because a rank only scores keys at or before its own queries, so early ranks sit idle for most of the rotation.
- **Striped [2]:** the sequence is split token-per-token assigned round-robin such that rank p takes every token with index $i \equiv p \pmod{P}$. Each rank then holds a uniform spread of tokens across the whole sequence, balancing the causal work. This prefilling allows to preserve contiguity.
- **Zigzag [6]:** round-robin still biases later ranks toward later tokens. Zigzag instead assigns the chunks in a mirrored order $(0, 1, \ldots, P-1, P-1, \ldots, 1, 0)$, so each rank gets an early chunk paired with its late reflection. This makes the causal balance exact and keeps chunks contiguous, simplifying the block-causal boundary check in our kernel.

Speed-up

The zigzag layout represents a theoretical speed-up of $\boxed{2 - \frac{1}{P}}$ against the contiguous layout in the causal setting. See a proof in Appendix C.

Communication topology. The standard flat ring schedule treats all network connections uniformly, which degrades performance on clusters characterized by severe bandwidth asymmetry. On the Stanford `gpu-turing` partition, intra-node communication achieves 6.2 GB/s, whereas inter-node traffic only achives 117 MB/s, which gives a $\sim 50\times$ bandwidth penalty.

To mitigate this hardware bottleneck, we designed a custom **Hierarchical 2D Ring Schedule**. We arrange the P processors into a grid of N nodes by g local GPUs. Key-value shards are rotated through fast intra-node rings using peer-to-peer `ncclSend`/`ncclRecv` streams and simultaneously

rotated between every nodes across the slow ethernet uplink. Animated figures illustrate this communication pattern.

Speed-up

The theoretical speed-up obtained by our custom schedule is equal to $\boxed{\frac{P-1}{P-g}}$. For instance, the speed-up equals 1.75 on a setup with 2 nodes with 4 GPUs each. See a proof in Appendix C.

3 Results and analysis

Correctness & verification

- ✓ Every kernel is validated against a PyTorch reference and an independent CPU oracle . Across all configurations (token layouts, causal and non-causal masks, ranks) the maximum element-wise error is 2.4×10^{-4}, under our 10^{-3} tolerance (set by the `fp16` transit floor).
- ✓ Because the online-softmax accumulation is invariant to the order in which K/V shards arrive, outputs are *bit-identical* regardless of ring size.
- ✓ The code base was developed test-first and continuous integration runs the full suite on every push and reports line coverage.

3.1 Benchmarking and instrumentation

Per-phase CUDA-event timers split every run into communication, compute, and exposed-wait time; Nsight Systems confirms the overlap timeline and Nsight Compute supplies the kernel roofline used in Section 3.2. The raw comm timer is contaminated by launch overhead and compute-coupled sync, so we report clean `cudaEvent` kernel time for compute and the analytic wire $(P-1)(\alpha' + \beta' n)$ from `comm_probe` for communication; the two agree with Nsight per-kernel busy time to $\sim 2\%$ (details in Section E).

3.2 Bottleneck analysis

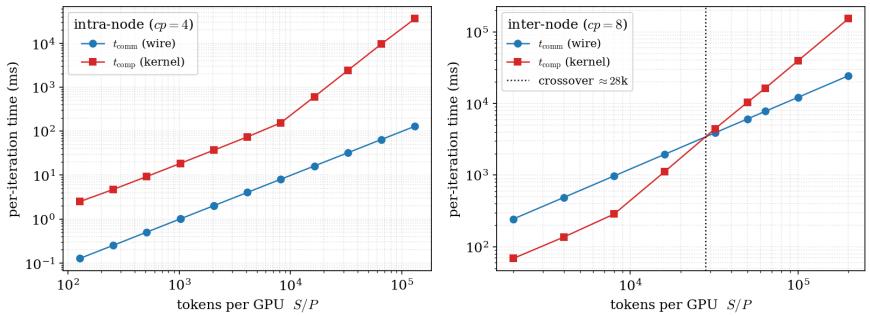


Figure 3: Communication vs. compute time as sequence length grows.

Sweeping S at fixed rank count tests the model's crossover prediction. Intra-node the kernel sits above the wire across the whole sweep while inter-node is communication-bound until the curves cross at $S/P \approx 3.2 \times 10^4$ tokens/GPU. The crossover's structure matches the model developed in

Section 2.2, but its location is an order of magnitude below the prediction. The cp1-calibrated $F^{\sigma}_{\text{eff}} = 23\,\text{TFLOP/s}$ collapses to $\approx 2\,\text{TFLOP/s}$ at large shards and substituting that rate moves the predicted crossover to $\approx 3.4 \times 10^4$, within 6% of measurement.

Multi-node. As a result of this, multi-node is **communication-bound** for most practical sequence lengths. The wall is mostly explained by the cluster components. The 117 MB/s link is 94% of the 1 GbE line rate (Table 2), NCCL falls back to TCP over `eth0` for want of any IB/RDMA, and extra sockets or buffers recover nothing.

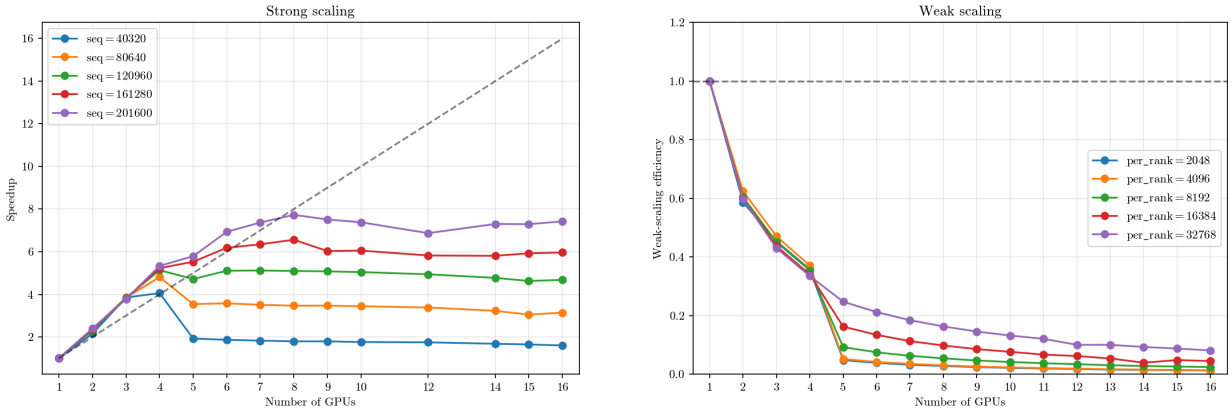
Intra-node. Within a single node communication is always hidden and the **kernel** is the limiter. Nsight Compute confirms it issues HMMA tensor-core instructions (57 TFLOP/s raw-equivalent at D=64, above the 32.6 TFLOP/s non-tensor ceiling). The remaining gap to the 130.5 TFLOP/s peak is non-GEMM work like online-softmax `exp` on the SFU, tile staging and rescaling as well as limited sm_75 occupancy.

3.3 Scalability

Strong and weak scaling are plotted at Figure 4 for different sequence lengths. As expected, longer sequence lengths (meaning more work) induce better scaling performance.

Overall, the scaling performance is heavily limited by the communications required by the algorithm. More noticeably, the intra-node communications necessary for 5+ ranks significantly hurt scaling performance.

Apart from pure performance, one crucial scalability advantage of the algorithm regards the ability to serve increasing sequence lengths as the number of processors increases. To this regard, the scaling is optimal and purely linear, as illustrated on Figure 10. Therefore adding more processors is mainly motivated by increasing maximum sequence length rather than obtaining better performance.



(a) Strong scaling

(b) Weak scaling

Figure 4: Speedup and parallel efficiency vs rank count.

3.4 Effect of algorithmic variants

Sequence partitions. Figure 5 reports the per-step speedup over the contiguous baseline at cp=4 (single node compute-bound), sweeping the zig-zag granularity n ($2n$ sub-chunks per rank).

Zigzag reaches $\approx 1.5\times$, just short of the $2 - 1/P = 1.75$ ceiling. The gap is the per-tile launch and staging overhead the work-count model omits. It corroborates Section 3.2 : the same non-GEMM cost that collapses F_{eff}^{σ} from 23 to $\approx 2\,\text{TFLOP/s}$ caps the realizable balance gain, so the kernel is overhead-limited, not balance-limited. Finer n only helps once the shard is large enough to amortize its extra per-chunk cost. Striped, despite balancing token count, runs $\approx 0.8\times$ *slower* than contiguous: its scattered chunks defeat the block-causal fast path, which is precisely why zigzag, contiguous within each chunk, is our choice.

Communication topology. Results are provided in Figure 6. We observe that the custom hierarchical layout we introduced consistently outperforms the standard ring layout across different sequence lengths. In addition, the magnitude of the speed-up across nodes and processors settings are consistent with the theoretical formula obtained in Section 2.3, as reported in Table 4.

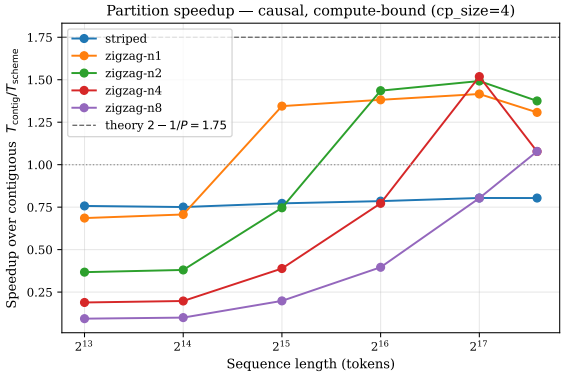


Figure 5: Per-step speedup of each partition layout over the contiguous baseline

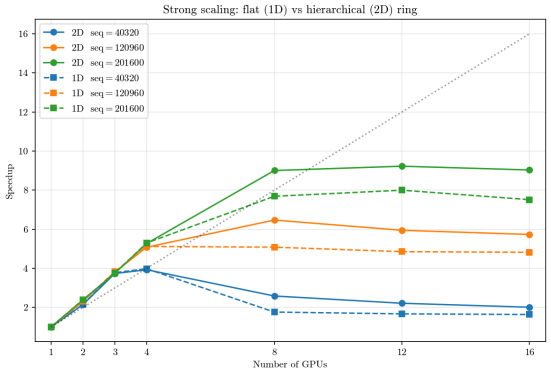


Figure 6: Strong scaling comparison of the 2D ring variant over the flat ring baseline

4 Discussion, limitations, and future work

Discussion. This work introduced a custom CUDA implementation of Ring Attention for sequence parallelism. The main results include a detailed comparative study of token layouts to allow better work repartition across different ranks, as well as a custom hierarchical grid topology tailoring the algorithm to the specific constraints of the class computing cluster. While improving the kernel and token layout lead to significant improvements of the compute time, introducing the custom 2D ring clearly gave the largest payoff by reducing the communication bottleneck.

Limitations. Because the hardware is lacking GPUDirect, falling back to TCP over a 1 GbE link caps multi-node efficiency. Ideally we would upgrade intra-node communications to faster components, like NVLink, to get past this physical bottleneck.

Future work. As mentioned earlier, the code base also includes an implementation of grouped-query attention, for which we decided not to provide an extensive analysis for the sake of conciseness. More generally, it is clear that modern attention implementations, such as Multi-Head Latent Attention (MLA) [4] are highly relevant to this setting where the memory footprint of the key/value pairs is critical.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, December 2023. arXiv:2305.13245 [cs.CL].

[2] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal transformers, 2023.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, June 2022. arXiv:2205.14135 [cs.LG].

[4] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruizhe Pan, Runxin Xu, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Size Zheng, T. Wang, Tian Pei, Tian Yuan, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Liu, Xin Xie, Xingkai Yu, Xinnan Song, Xinyi Zhou, Xinyu Yang, Xuan Lu, Xuecheng Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Zheng, Yichao Zhang, Yiliang Xiong, Yilong Zhao, Ying He, Ying Tang, Yishi Piao, Yixin Dong, Yixuan Tan, Yiyuan Liu, Yongji Wang, Yongqiang Guo, Yuchen Zhu, Yuduan Wang, Yuheng Zou, Yukun Zha, Yunxian Ma, Yuting Yan, Yuxiang You, Yuxuan Liu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhewen Hao, Zhihong Shao, Zhiniu Wen, Zhipeng Xu, Zhongyu Zhang, Zhuoshu Li, Zihan Wang, Zihui Gu, Zilin Li, and Ziwei Xie. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, June 2024. arXiv:2405.04434 [cs.CL].

[5] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring Attention with Blockwise Transformers for Near-Infinite Context.

[6] Nouamane Tazi, Ferdinand Mom, Haojun Zhao, Phuc Nguyen, Mohamed Mekkouri, Leandro von Werra, and Thomas Wolf. *The Ultra-Scale Playbook: Training LLMs on GPU Clusters*. Hugging Face Press, 2025.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023. arXiv:1706.03762 [cs.CL].

[8] Phil Wang. lucidrains/ring-attention-pytorch, May 2026. original-date: 2024-02-14T14:37:51Z.

[9] Zilin Zhu. zhuzilin/ring-flash-attention, June 2026. original-date: 2024-02-21T08:04:32Z.

Contents

1 Introduction **1**

2 Approach **1**
2.1 Parallelization strategy (CUDA + MPI) . . . 1
2.2 Performance model . . . 2
2.3 Algorithmic variants explored . . . 3

3 Results and analysis **4**
3.1 Benchmarking and instrumentation . . . 4
3.2 Bottleneck analysis . . . 4
3.3 Scalability . . . 5
3.4 Effect of algorithmic variants . . . 5

4 Discussion, limitations, and future work **6**

A Computing cluster setup and information **8**

B Kernel implementation details **9**

C Theoretical speed-up of algorithmic variants **10**

D Profiling **10**

E Additional figures **11**

A Computing cluster setup and information

All experiments run on the Stanford HPCC `gpu-turing` partition. Table 1 lists the hardware inventory; Table 2 gives the constants used by the performance model of Section 2.2.

Resource	Configuration
Nodes	5 (`hpcc-gpu-5-[1-5]`)
GPUs per node	4
GPU	NVIDIA Quadro RTX 6000
GPU memory	24 GB GDDR6, 672 GB/s peak
Host per node	16 CPU cores, 128 GB RAM
Intra-node link	PCIe Gen3 ×16 (no NVLink)
Inter-node link	single 1 GbE uplink (no InfiniBand / RDMA)

Table 1: `gpu-turing` hardware inventory.

Symbol	Meaning	Value	Source
σ	bytes / element (FP16 transit)	2 B	design choice
g	GPUs per node	4	hardware
D, H	head dim, number of heads	$\{32, 64, 128, 256\}$, 4	benchmark config
α	intra-node latency	≈ 0.35 μs	comm_probe (MPI shared-mem)
β	intra-node inv-bandwidth	1.6×10^{-10} s/B = 6.2 GB/s	comm_probe (MPI shared-mem)
α'	inter-node latency	≈ 44 μs (small-msg floor)	comm_probe (eth0 TCP)
β'	inter-node inv-bandwidth	8.5×10^{-9} s/B = 117 MB/s	comm_probe (eth0 TCP)
F^{σ}_{peak}	FP16 tensor-core peak	130.5 TFLOP/s	RTX 6000 spec
F^{σ}_{eff}	effective attention rate	≈ 23 (D=128), 29 (D=64) TFLOP/s	cp1 bench (useful causal FLOPs)
B_{DRAM}	GDDR6 bandwidth	672 peak / 432 achieved GB/s	spec / Nsight Compute

Table 2: Measured constants for the performance model. Communication constants come from a host-buffer MPI ping-pong with an $\alpha + \beta n$ least-squares fit (`scripts/comm_probe`).

B Kernel implementation details

Each ring step is one kernel templated on the head dimension $D \in \{32, 64, 128, 256\}$, launched as $(\lceil S_q/16 \rceil \times H \times B)$ blocks of a single warp: a block owns a 16-row query tile and walks the arrived K/V shard in 16-key tiles (Algorithm 1). Both matmuls use 16×16×16 wmma Tensor-Core fragments, fp16 in and fp32 accumulate; loading K column-major gives $K^\top$ for free. The whole working set (Q/K/V tiles, scores, the running O) stays in shared memory, about 22 KB at D=128, under the 48 KB sm_75 limit.

Algorithm 1 One ring-step kernel (per block: one warp, a 16-row query tile)

load Q tile and state (O, m, ℓ) from global into shared memory
for each 16-key tile of the arrived K/V shard **do**
  load K, V into shared memory (fp16)
  $S \leftarrow \frac{1}{\sqrt{D}} QK^\top$ ▷ wmma, fp32 accumulate
  barrier
  for query rows $r = 0 \ldots 15$ (one thread each) **do**
    mask $S_{r,:}$ from global causal positions
    $m' \leftarrow \max(m_r,\ \max_j S_{r,j})$; $\alpha \leftarrow e^{\,m_r - m'}$
    $P_{r,:} \leftarrow e^{\,S_{r,:} - m'}$
    $\ell_r \leftarrow \alpha\, \ell_r + \sum_j P_{r,j}$; $O_{r,:} \leftarrow \alpha\, O_{r,:}$; $m_r \leftarrow m'$
  end for
  barrier
  $O \leftarrow O + PV$ ▷ wmma, fp32 accumulate
end for
write (O, m, ℓ) back to global

Because O lives in shared memory rather than in a wmma fragment, the softmax runs as plain CUDA on 16 threads between the matmuls and rescales each row of O in place by $\alpha = e^{\,m_r - m'}$; this per-row exp on the SFU is the non-GEMM cost seen in the roofline of Section 3.2. The mask is read from global token positions, so one kernel serves the contiguous, striped and zigzag layouts. The kernel is also stateful across the ring: each step reads (O, m, ℓ) from global, folds in one shard on a compute stream while the next is exchanged on a copy stream, and writes the fp32 statistics back, with a finalize pass dividing O by ℓ at the end.

C Theoretical speed-up of algorithmic variants

Sequence partitions. In the causal setting, the total amount of work is proportional to $S(S-1)/2 := W$. The contiguous layout would assign

$$\sum_{t=\frac{P-1}{P}S}^{S} t \approx W\left[1-\left(\frac{P-1}{P}\right)^2\right] \approx W\left(\frac{2}{P}-\frac{1}{P^2}\right)$$

units of work to the last rank, which is the bottleneck. On the contrary, the zigzag layout assigns W/P units of work to every rank, which yields a theoretical speed-up [2] over the naïve layout of :

$$\boxed{2-\frac{1}{P}.}$$

Communication topology. In the flat ring schedule, each step is bounded by the time T of one key/value shard transfer across the inter-node links, which gives a total of time of $(P-1)T$.

In the hierarchical schedule, the total runtime[3] is built as $\frac{P}{g}-1$ rounds with g shards sent across the inter-node links at each step. This gives a total time of $(P-g)T$.

The theoretical speed-up obtained by our custom schedule is therefore equal to $\boxed{\frac{P-1}{P-g}}$. For instance, the speed-up equals 1.75 on a setup with 2 nodes with 4 GPUs each.

D Profiling

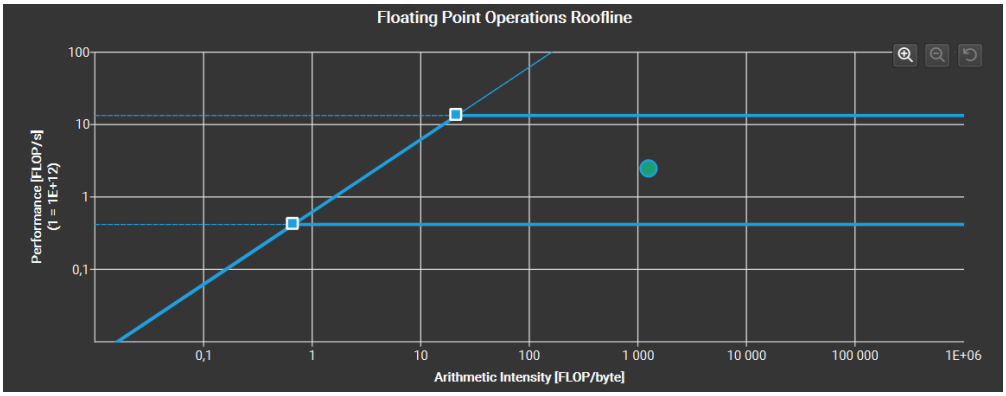


Figure 7: Roofline model of our kernel with head dim = 64

On figure 9, we can see that while the K/V shards are being sent to the next node, the node does 4 attention steps with 3 NCCLsend/receive, this corresponds to the inner loop. We see that the main bound is the outer loop communication between nodes.

[2] **Warning :** obviously this speed-up is observed in practice only when the entire process is compute-bound.

[3] Rigourously, the total runtime also includes the runtime of the last intra-node ring rotation, which is an order of magnitude smaller because of the fast intra-node communications.

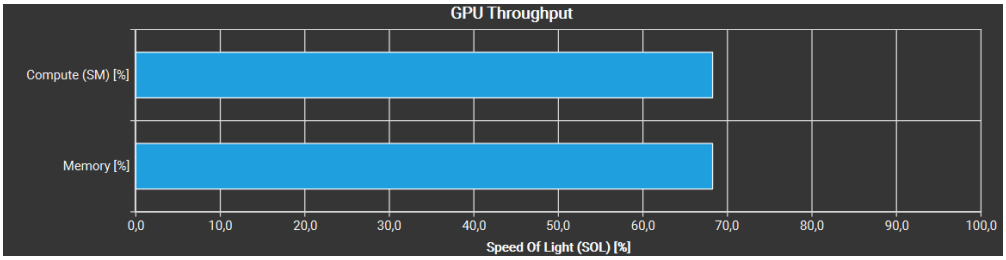


Figure 8: GPU Throughput chart for kernel with head dim = 64

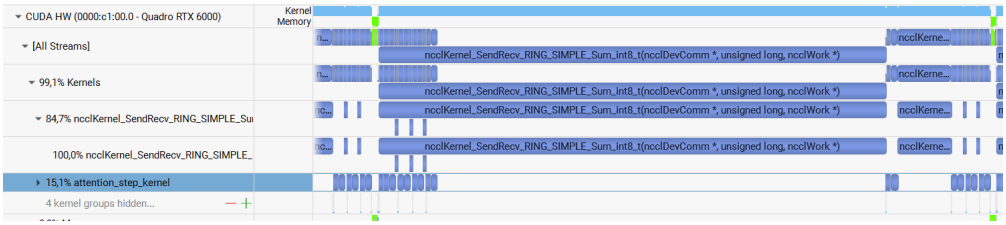


Figure 9: Nsight System screenshot of the 2D hierarchical ring, using NCCL

E Additional figures

Communication timer. Our per-phase timer brackets the NCCL op on the copy stream, so it absorbs three things that are not wire time: kernel-launch overhead, the `fp16`↔`fp32` widen kernels, and a compute-coupled synchronisation that makes the measured comm window spuriously track compute rather than the network. We therefore never quote it directly. Instead compute is taken from clean `cudaEvent` kernel time and communication from the analytic wire $(P-1)(\alpha' + \beta' n)$ calibrated by `comm_probe`; both agree with Nsight Systems per-kernel busy time to $\sim$2%, which is the basis for the numbers in Section 3.1.

On figure 10, the maximum sequence length scales linearly with the number of processors

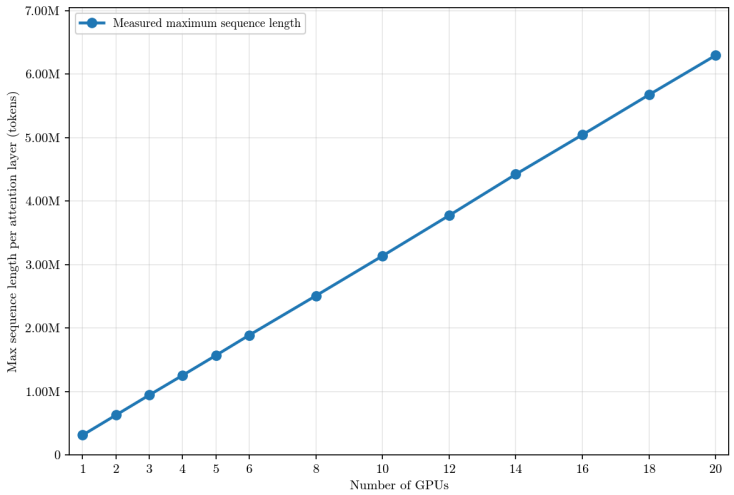


Figure 10: Maximum sequence length scaling with number of processors.

Back-end	comm (ms)	comp (ms)	wait (ms)	**total (ms)**
NCCL (P2P/IPC + Socket)	506	139	0	523
MPI (host-staged)	14	141	632	655

Table 3: Iso-byte comparison of NCCL communications against MPI communications.

cp	N	G	Theory	meas. S=40320	meas. S=120960	meas. S=201600
8	2	4	1.75	1.46	1.27	1.17
12	3	4	1.38	1.32	1.23	1.15
16	4	4	1.25	1.23	1.19	1.20

Table 4: Theoretical and measured speed-ups between 2D ring and flat ring (causal+zigzag, D=128, H=8).